Deployment & Update Guide — DeoDap Care Panel (Email Automation)

How this project is deployed live, and exactly what to do when you change the code. This reflects the **real** production setup (not a generic guide).

---

1. What's live, and where

Item	Value
Live URL	https://care.deodap.info/email_automation
Server (SSH)	`ssh -i "rdapp.pem" ubuntu@43.204.203.9`
App folder	`/var/www/Email_Automation`
GitHub repo	`https://github.com/mayurkhaniyadeodap-cyber/email_automation_project`
Backend	Django + gunicorn, systemd service `deodap-care`, on `127.0.0.1:8013`
Frontend	React (Vite build) served by nginx from `Web/frontend/dist/`
Web server	nginx (site `care.deodap.info`, HTTPS via Let's Encrypt)
Background jobs	cron — `fetch_emails` every 5 min, `sweep_waiting` every 30 min
Email	Gmail over IMAP (fetch) + SMTP (replies), 16-char app password

> ⚠️ This is a **shared server** — it also runs other apps (the `/support/` ticket app, `vastate.in`, etc.). Don't touch their nginx sites, ports (8001/4000), or systemd services. Our app only uses port **8013** and the `/email_automation` path.

Architecture

```
Browser
  └─ https://care.deodap.info/email_automation/ ─► nginx (443, TLS)
        ├─ /email_automation/                 → React SPA (Web/frontend/dist)
        ├─ /email_automation/api/    ──strip─► gunicorn 127.0.0.1:8013  (Django API)
        ├─ /email_automation/admin/ ──strip─► gunicorn 127.0.0.1:8013  (Django admin)
        └─ /email_automation/static/          → Web/backend/staticfiles
```

nginx strips the `/email_automation` prefix before proxying; Django re-adds it to links/redirects via `FORCE_SCRIPT_NAME`. The React app is built with `VITE_BASE=/email_automation/` so its assets and API calls use the same prefix.

---

2. ⭐ Updating after you change the code (the everyday flow)

You edit code locally → push to GitHub → pull + rebuild on the server.

Step A — Push your changes (local machine)

```
cd "c:/Users/Deo Dap/deodap-care"
git add -A
git commit -m "describe your change"
git push origin main
```

Step B — Deploy on the server

```
ssh -i "rdapp.pem" ubuntu@43.204.203.9
cd /var/www/Email_Automation
git pull origin main
```

Then run **only the part that matches what you changed**:

If you changed BACKEND code (Python / Django):

```
cd /var/www/Email_Automation/Web/backend
source venv/bin/activate
pip install -r requirements.txt          # only if requirements changed
python manage.py migrate                 # only if you added/changed models
python manage.py collectstatic --noinput # only if admin/static changed
sudo systemctl restart deodap-care
```

If you changed FRONTEND code (React / Vite):

```
cd /var/www/Email_Automation/Web/frontend
npm install                              # only if package.json changed
VITE_BASE=/email_automation/ npm run build   # ⚠️ MUST pass VITE_BASE or links break
# no restart needed — nginx serves the new dist/ immediately
```

> 🔴 **Always** build the frontend with `VITE_BASE=/email_automation/`. Plain `npm run build` produces a root-path build whose assets/links 404 under the sub-path.

If you changed BOTH — do backend then frontend, then:

```
sudo systemctl restart deodap-care && sudo systemctl reload nginx
```

Step C — After deploying frontend, hard-refresh the browser

`Ctrl + Shift + R` — the old JS bundle is cached and will otherwise keep running.

One-liner (full update — safe to always use)

```
cd /var/www/Email_Automation && git pull origin main && \
cd Web/backend && source venv/bin/activate && pip install -r requirements.txt && \
python manage.py migrate && python manage.py collectstatic --noinput && \
cd ../frontend && npm install && VITE_BASE=/email_automation/ npm run build && \
sudo systemctl restart deodap-care && sudo systemctl reload nginx
```

---

3. First-time setup (already done — reference only)

If the app ever has to be set up from scratch on a fresh server:

```
# 1. System packages
sudo apt update && sudo apt install -y python3-venv python3-pip nginx git
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash - && sudo apt install -y nodejs

# 2. Code
cd /var/www && git clone https://github.com/mayurkhaniyadeodap-cyber/email_automation_project.git Email_Automation

# 3. Backend
cd /var/www/Email_Automation/Web/backend
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env && nano .env        # fill REAL values (see §4)
python manage.py migrate
python manage.py collectstatic --noinput

# 4. Frontend
cd ../frontend && npm install && VITE_BASE=/email_automation/ npm run build

# 5. gunicorn service  (127.0.0.1:8013)
#   create /etc/systemd/system/deodap-care.service (WorkingDirectory=.../Web/backend,
#   ExecStart=.../venv/bin/gunicorn deodap_care.wsgi:application --bind 127.0.0.1:8013 --workers 3)
sudo systemctl daemon-reload && sudo systemctl enable --now deodap-care

# 6. nginx — add these location blocks inside the care.deodap.info :443 server:
#   location = /email_automation { return 301 /email_automation/; }
#   location /email_automation/api/    { proxy_pass http://127.0.0.1:8013/api/;   ...proxy headers... }
#   location /email_automation/admin/  { proxy_pass http://127.0.0.1:8013/admin/; ...proxy headers... }
#   location /email_automation/static/ { alias /var/www/Email_Automation/Web/backend/staticfiles/; }
#   location /email_automation/        { alias /var/www/Email_Automation/Web/frontend/dist/;
#                                        try_files $uri $uri/ /email_automation/index.html; }
sudo nginx -t && sudo systemctl reload nginx

# 7. Background jobs
crontab -e   # add the two lines from §5
```

One-time database setup (required, else the panel is empty)

```
cd /var/www/Email_Automation/Web/backend && source venv/bin/activate

# a) admin user (auto-login needs a user named "admin" / a superuser)
python manage.py shell -c "from django.contrib.auth import get_user_model as g; U=g(); \
u,_=U.objects.get_or_create(username='admin', defaults={'is_staff':True,'is_superuser':True}); \
u.is_staff=u.is_superuser=u.is_active=True; u.set_password('CHANGE-ME'); u.save(); print('ok')"

# b) Organization -> Brand -> Mailbox (fetch needs a Mailbox row, else FETCH MAIL = 404)
python manage.py shell -c "from apps.organizations.models import Organization,Brand,Mailbox as M; \
o,_=Organization.objects.get_or_create(name='DeoDap'); b,_=Brand.objects.get_or_create(organization=o,name='DeoDap'); \
M.objects.get_or_create(email_address='chintandeodap2134@gmail.com', defaults={'brand':b,'provider':'imap','is_active':T

# c) seed the 16-category support taxonomy (else Settings -> Categories & rules is empty)
python manage.py seed_taxonomy --all
```

---

4. Environment variables (`Web/backend/.env`)

Live on the server only — **never commit** `.env`. Key values for this deployment:

```
DJANGO_DEBUG=False
DJANGO_SECRET_KEY=<long random string>
DJANGO_ALLOWED_HOSTS=care.deodap.info,43.204.203.9,localhost
CORS_ALLOWED_ORIGINS=https://care.deodap.info
CSRF_TRUSTED_ORIGINS=https://care.deodap.info
FORCE_SCRIPT_NAME=/email_automation
PUBLIC_BASE_URL=https://care.deodap.info/email_automation

EMAIL_PROVIDER=imap
IMAP_HOST=imap.gmail.com
IMAP_USER=chintandeodap2134@gmail.com
IMAP_PASSWORD=<16-char Gmail app password, no spaces>
SMTP_HOST=smtp.gmail.com           # SMTP login reuses IMAP_USER / IMAP_PASSWORD
# + real GROQ_API_KEY / GEMINI_API_KEY / SHOPIFY_* / CARE_PANEL_* values
```

After editing `.env`, restart the backend: `sudo systemctl restart deodap-care`.

---

5. Background jobs (cron)

gunicorn does **not** run the in-process scheduler, so mail fetch comes from cron:

```
*/5  * * * * cd /var/www/Email_Automation/Web/backend && venv/bin/python manage.py fetch_emails  >> /home/ubuntu/care-fe
*/30 * * * * cd /var/www/Email_Automation/Web/backend && venv/bin/python manage.py sweep_waiting >> /home/ubuntu/care-sw
```

Edit with `crontab -e`; view logs with `tail -f /home/ubuntu/care-fetch.log`.

---

6. Common operations

```
# Service
sudo systemctl status deodap-care          # is it running?
sudo systemctl restart deodap-care         # restart after backend/.env change
journalctl -u deodap-care -f               # live backend logs

# nginx
sudo nginx -t && sudo systemctl reload nginx

# Pull mail right now (manual)
cd /var/www/Email_Automation/Web/backend && venv/bin/python manage.py fetch_emails

# Test outbound email
venv/bin/python manage.py test_smtp --to you@example.com
```

---

7. Troubleshooting (issues we actually hit)

Symptom	Cause & fix
404 Not Found at `/email_automation` (no slash)	nginx needs `location = /email_automation { return 301 /email_automation/; }`
Blank page / assets 404 after deploy	Frontend built without `VITE_BASE=/email_automation/` — rebuild with it
Old behavior after deploy	Browser cached the old JS — hard-refresh `Ctrl+Shift+R`
Attachments "Cannot GET /api/attachments/.."	URL missing the `/email_automation` prefix — fixed by `attachmentUrl()` helper (rebuild frontend)
FETCH MAIL → 404 "No mailbox"	No `Mailbox` row in DB — run the §3(b) setup
Settings → Categories & rules empty	Taxonomy not seeded — `python manage.py seed_taxonomy --all`
Panel won't open / "No auto-login user"	No `admin` superuser — run the §3(a) setup
`ingested 0` **on fetch**	Normal if no **unread** mail (first fetch pulls UNSEEN only)
502 Bad Gateway	`sudo systemctl status deodap-care` ; check `journalctl -u deodap-care -f`
DisallowedHost	Add the host to `DJANGO_ALLOWED_HOSTS` in `.env` , restart service

8. Notes

- **Database:** SQLite (`Web/backend/db.sqlite3`) — fine for low volume on one server.
- **Secrets:** `.env` , `Credentials.json` , `rdapp.pem` are git-ignored — keep them off GitHub.
- `care.deodap.in` **vs** `care.deodap.info` **:** they are **different servers**. This app is on `care.deodap.info` (43.204.203.9). `care.deodap.in` (used by `build_tracking_url` for customer tracking links) lives on a separate machine.